

Algoritmos en R — Guía Práctica

Juan Marcos Aguirre Simionato

2026-04-01

Contents

1. Primeros pasos en R	1
2. Vectores y operaciones vectoriales	1
3. Matrices y Álgebra Lineal	2
4. Funciones definidas por el usuario	2
5. Control de flujo: bucles y condicionales	3
6. Datasets internos y estadística descriptiva	3
7. Visualización de datos	4
8. Regresión lineal — Resultados completos	6
9. Funciones matemáticas útiles	7
10. Tips rápidos de R	7

1. Primeros pasos en R

R permite operar con variables escalares, vectores y matrices de forma muy similar a MATLAB.

```
A <- 40
B <- 60
C <- B - A
C

## [1] 20

# Operaciones básicas
x <- 2^10
raiz <- sqrt(144)
cat("2^10 =", x, " | sqrt(144) =", raiz)

## 2^10 = 1024 | sqrt(144) = 12
```

2. Vectores y operaciones vectoriales

```
v <- c(1, 4, 9, 16, 25)
cat("Vector:      ", v, "\n")

## Vector:      1 4 9 16 25

cat("Raíz c/elem: ", sqrt(v), "\n")

## Raíz c/elem: 1 2 3 4 5

cat("Suma:        ", sum(v), "\n")
```

```
## Suma:          55
cat("Media:      ", mean(v), "\n")

## Media:         11
cat("Desv. Est.: ", sd(v), "\n")

## Desv. Est.:    9.66954
```

3. Matrices y Álgebra Lineal

Muy útil para sistemas de ecuaciones en ingeniería.

```
# Definición
A <- matrix(c(2, 1, 5, 3), nrow = 2, byrow = TRUE)
b <- c(8, 17)

cat("Matriz A:\n"); print(A)

## Matriz A:
##      [,1] [,2]
## [1,]    2    1
## [2,]    5    3
cat("Vector b:", b, "\n")

## Vector b: 8 17

# Solución del sistema Ax = b
x <- solve(A, b)
cat("Solución x:", x, "\n")

## Solución x: 7 -6

# Determinante e inversa
cat("Det(A):", det(A), "\n")

## Det(A): 1

cat("Inversa de A:\n"); print(solve(A))

## Inversa de A:
##      [,1] [,2]
## [1,]    3   -1
## [2,]   -5    2
```

4. Funciones definidas por el usuario

```
# Función genérica
potencia <- function(base, exp) {
  return(base^exp)
}

# Función de ingeniería: velocidad con MRUA
```

```

velocidad <- function(v0, a, t) {
  v0 + a * t
}

cat("2^8 =", potencia(2, 8), "\n")

## 2^8 = 256

cat("v(t=5) con v0=0, a=9.8 m/s^2 →", velocidad(0, 9.8, 5), "m/s\n")

## v(t=5) con v0=0, a=9.8 m/s^2 → 49 m/s

```

5. Control de flujo: bucles y condicionales

```

# Serie de Fibonacci
fib <- numeric(10)
fib[1] <- 1; fib[2] <- 1
for (i in 3:10) {
  fib[i] <- fib[i-1] + fib[i-2]
}
cat("Fibonacci (10 términos):", fib, "\n")

## Fibonacci (10 términos): 1 1 2 3 5 8 13 21 34 55

# Clasificador con if/else
clasificar <- function(x) {
  if (x > 0) "positivo" else if (x < 0) "negativo" else "cero"
}
cat("Clasificación de -7:", clasificar(-7), "\n")

## Clasificación de -7: negativo

```

6. Datasets internos y estadística descriptiva

Con `data()` en la consola podés ver todos los datasets disponibles en R.

```

# Dataset 'mtcars': autos con variables técnicas
summary(mtcars[, c("mpg", "hp", "wt")])

```

```

##      mpg          hp          wt
##  Min.   :10.40   Min.   : 52.0   Min.   :1.513
## 1st Qu.:15.43   1st Qu.: 96.5   1st Qu.:2.581
##  Median :19.20   Median :123.0   Median :3.325
##   Mean  :20.09   Mean  :146.7   Mean   :3.217
## 3rd Qu.:22.80   3rd Qu.:180.0   3rd Qu.:3.610
##   Max.  :33.90   Max.  :335.0   Max.   :5.424

```

```

# Tabla de frecuencias
table(mtcars$cyl)

```

```

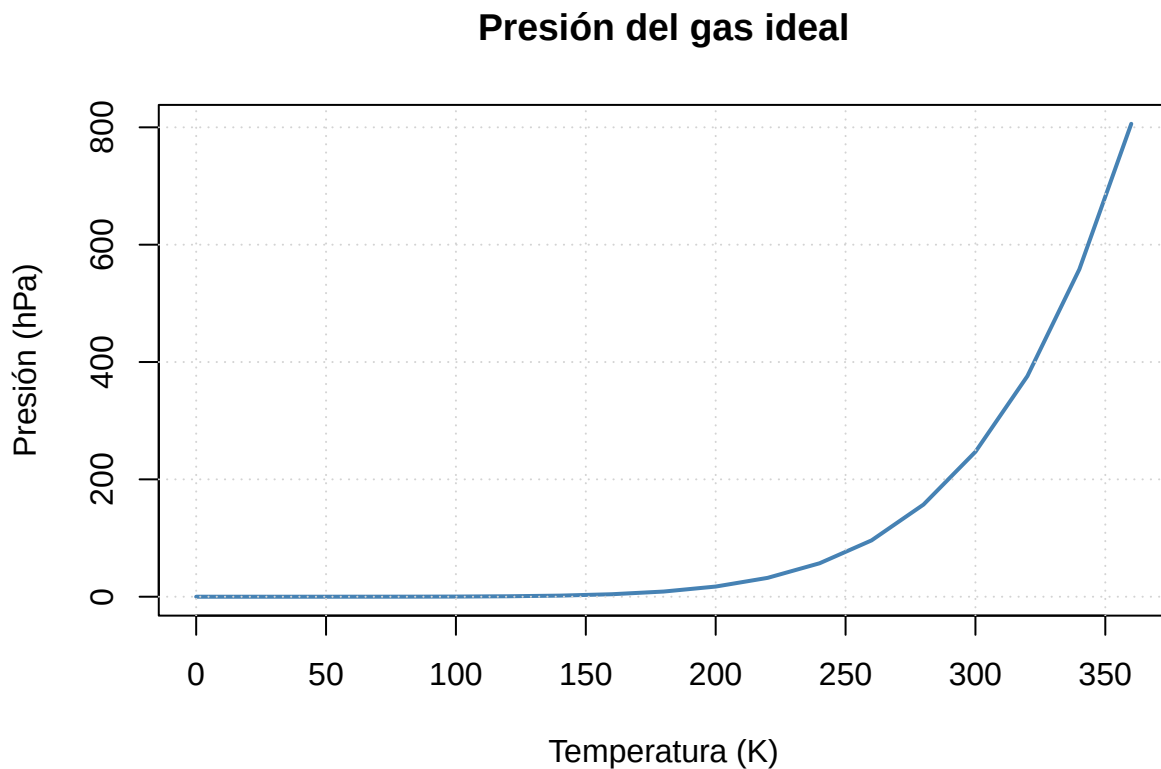
##
##  4  6  8
## 11  7 14

```

7. Visualización de datos

7.1 Gráfico base: presión vs temperatura

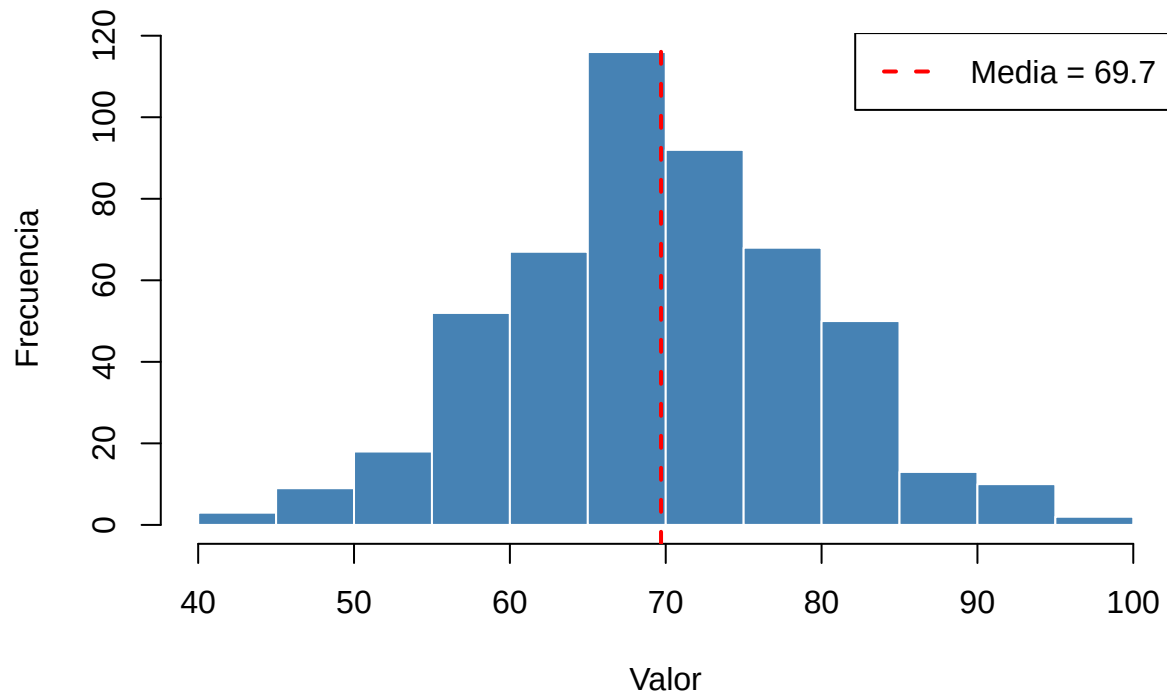
```
plot(pressure,  
     type = "l",  
     main = "Presión del gas ideal",  
     ylab = "Presión (hPa)",  
     xlab = "Temperatura (K)",  
     col = "steelblue",  
     lwd = 2)  
grid()
```



7.2 Histograma de una distribución normal

```
set.seed(42)  
datos <- rnorm(500, mean = 70, sd = 10)  
  
hist(datos,  
     breaks = 20,  
     col = "steelblue",  
     border = "white",  
     main = "Distribución Normal (=70, =10)",  
     xlab = "Valor",  
     ylab = "Frecuencia")  
abline(v = mean(datos), col = "red", lwd = 2, lty = 2)  
legend("topright", legend = paste("Media =", round(mean(datos), 2)),  
     col = "red", lty = 2, lwd = 2)
```

Distribución Normal ($\mu=70$, $\sigma=10$)



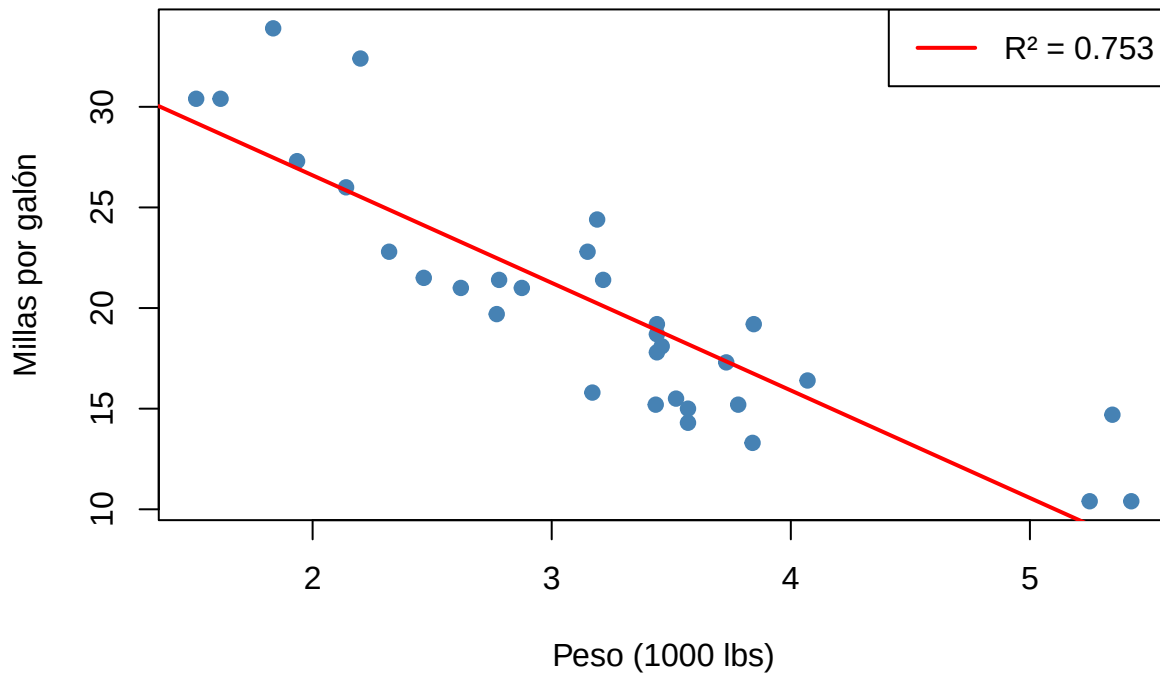
7.3 Gráfico de dispersión con regresión lineal

```
x <- mtcars$wt
y <- mtcars$mpg

plot(x, y,
     main = "Peso vs Consumo (mtcars)",
     xlab = "Peso (1000 lbs)",
     ylab = "Millas por galón",
     pch = 19,
     col = "steelblue")

modelo <- lm(y ~ x)
abline(modelo, col = "red", lwd = 2)
legend("topright",
     legend = paste("R² =", round(summary(modelo)$r.squared, 3)),
     col = "red", lwd = 2)
```

Peso vs Consumo (mtcars)



8. Regresión lineal — Resultados completos

```
summary(modelo)
```

```
##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.5432 -2.3647 -0.1252  1.4096  6.8727
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  37.2851     1.8776   19.858 < 2e-16 ***
## x            -5.3445     0.5591   -9.559 1.29e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3.046 on 30 degrees of freedom
## Multiple R-squared:  0.7528, Adjusted R-squared:  0.7446
## F-statistic: 91.38 on 1 and 30 DF,  p-value: 1.294e-10
```

9. Funciones matemáticas útiles

```
# Derivada numérica (diferencia finita)
f <- function(x) x^3 - 2*x + 1
df <- function(x) (f(x + 1e-5) - f(x)) / 1e-5
cat("f'(2) numérico :", round(df(2), 4), "\n")

## f'(2) numérico : 10.0001

cat("f'(2) analítico :", 3*2^2 - 2, "\n") # 3x2-2

## f'(2) analítico : 10

# Integral numérica
integral <- integrate(f, lower = 0, upper = 3)
cat(" ∫03 f(x)dx =", round(integral$value, 4), "\n")

## ∫03 f(x)dx = 14.25
```

10. Tips rápidos de R

Tarea	Comando
Ayuda sobre función	?nombre_función
Ver datasets disponibles	data()
Limpiar entorno	rm(list = ls())
Instalar paquete	install.packages()
Cargar paquete	library()
Ver estructura de objeto	str(objeto)
Ver primeras filas	head(datos)